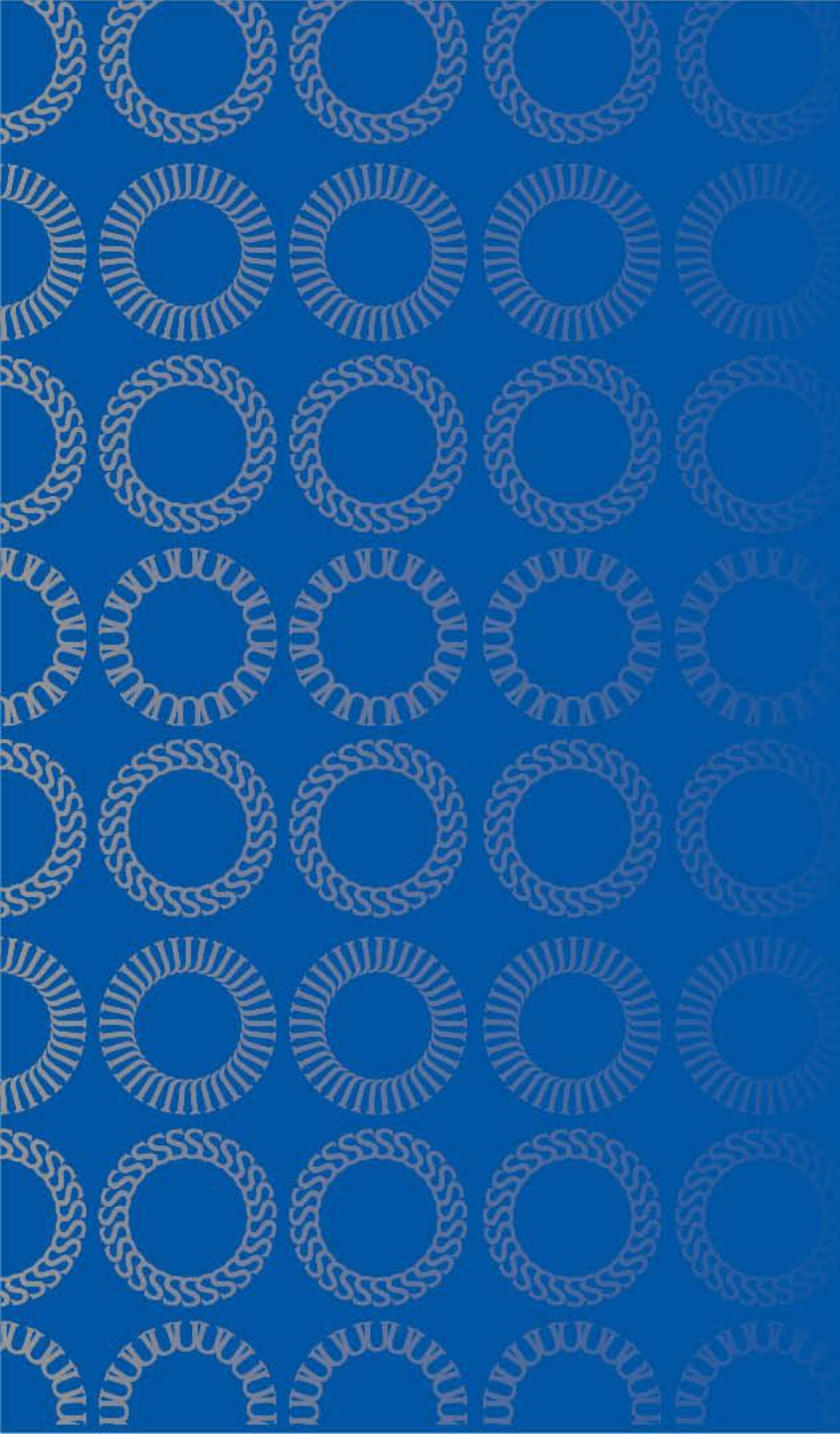




DATA 255 Deep Learning Technologies

Sept 4, 2025

Taehee Jeong, Ph.D.

Today's Topics

- Activation Functions in Neural Networks
- Initialize Weights
- Forward Pass
- Loss function

Why we need activation functions in NN?

- Activation functions in neural networks introduce **non-linearity**, enabling the network to learn complex patterns by transforming the weighted sum of inputs into an output signal.
- They determine whether a neuron should be activated (and thus contribute to the network's output) based on the input it receives.
- Without activation functions, neural networks would only be capable of learning linear relationships

Activation functions in Neural Network

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- ELU
- Softmax

Sigmoid

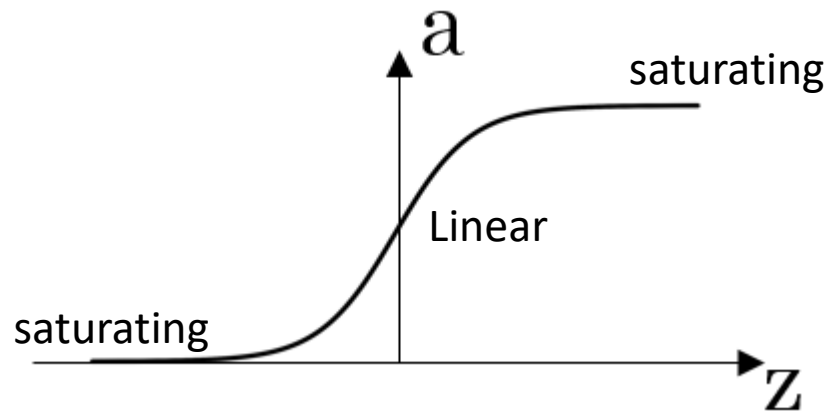


image: Deep learning, Andrew Ng

$$a = \sigma(z) = \frac{1}{1 + \exp(-z)}$$

- Sigmoid neurons can be saturated when z is very large or very small.
- The saturation leads vanishing gradients.
- The output is not zero-centered (0~1).
- $\exp()$ is computationally expensive.

tanh

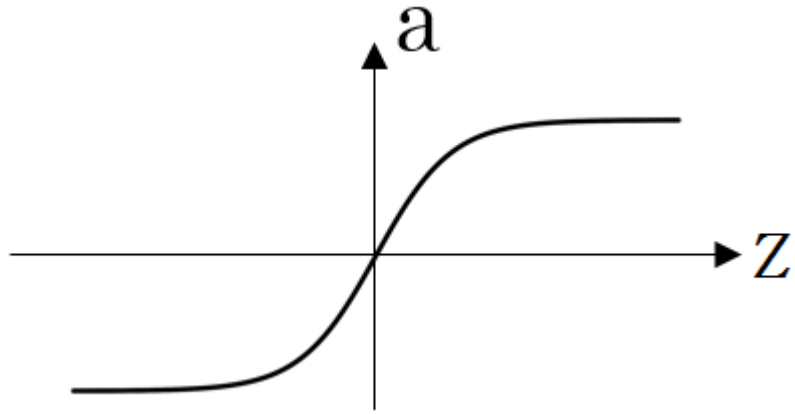


image: Deep learning, Andrew Ng

$$a = \tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

- The output is zero-centered(-1~+1).
- It is simply a scaled sigmoid function.

$$\tanh(z) = 2\sigma(2z) - 1$$

ReLU (Rectified Linear Unit)

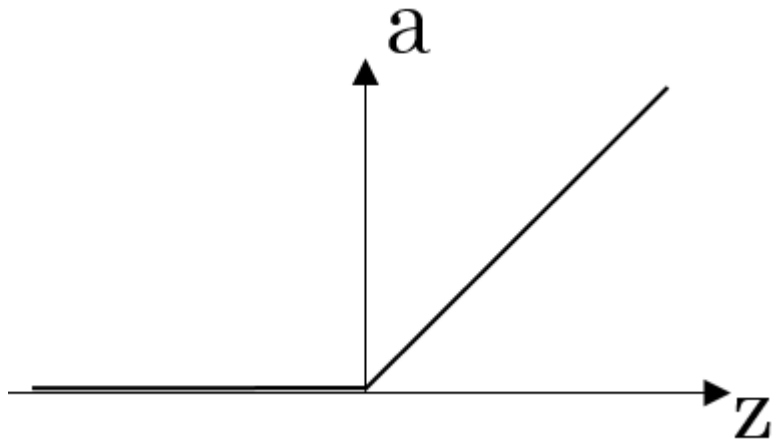


image: Deep learning, Andrew Ng

$$a = \text{ReLU}(z) = \max(0, z)$$

- It is zero when z is ≤ 0
- It is linear with slope 1 when z is > 0 .
- Fast to compute
- Gradients do not vanish for $z > 0$.
- Faster convergence in practice.
- Not zero-centered.
- It is most used in CNN.
- Some neurons can die if it is not activated, never updated.

Leaky ReLU

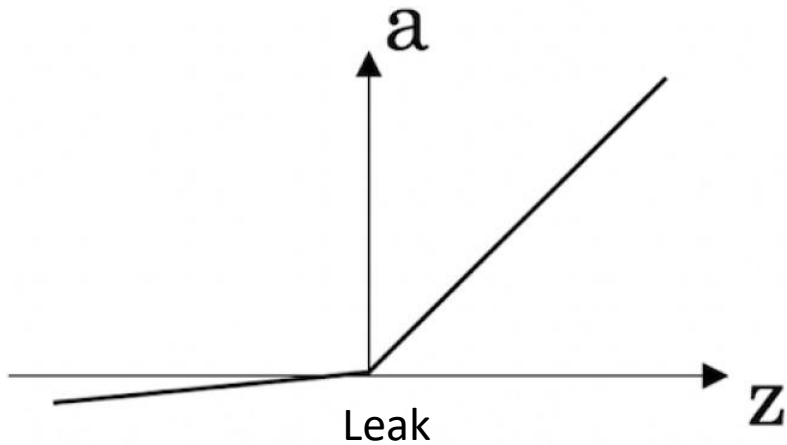


image: Deep learning, Andrew Ng

$$a = \text{Leaky ReLU}(z) = \max(0.01z, z)$$

- It is linear with small negative slope when z is ≤ 0
- It is linear with slope 1 when z is > 0 .
- neurons will not die

ELU(Exponential Linear Unit)

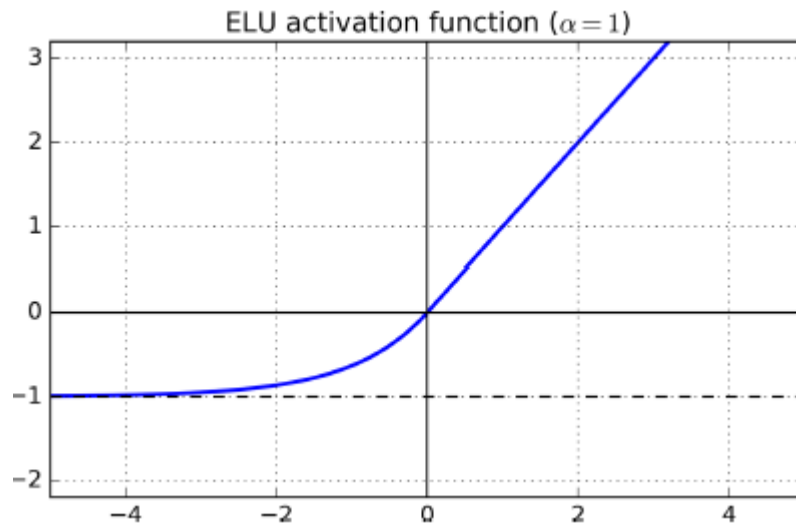


image: Hands-On ML, Aurelien Geron

$$a = ELU(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

- Nonzero gradient for $z < 0 \rightarrow$ avoid dying neurons issue
- It is smooth everywhere including $z=0$

Softmax score

When given an instance $\mathbf{x}$, the Softmax classifier first computes a score $s_k(\mathbf{x})$ for each class k , then estimates the probability of each class by applying the *softmax function* to the scores.

Softmax score for class k

$$s_k(x) = W^k \cdot x$$

$$p_k = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}$$

Softmax classifier

$$p_k = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}$$

- Multiclass classifier
- Provides normalized class probabilities

$$\hat{y} = \operatorname{argmax}_k p_k$$

It predicts the class with the highest estimated probability.

Xavier initialization

When using sigmoid activation function

Multiplies weights by $\frac{\sqrt{2}}{\sqrt{n_{inputs} + n_{outputs}}}$

Source: Hands-On ML, Aurelien Geron

Initialization for Hyperbolic tangent

When using tanh activation function

Multiplies weights by $4 \frac{\sqrt{2}}{\sqrt{n_{inputs} + n_{outputs}}}$

Source: Hands-On ML, Aurelien Geron

He initialization

When using ReLU activation function (and its variants, including the ELU activation)

Multiplies weights by $\sqrt{2} \frac{\sqrt{2}}{\sqrt{n_{inputs} + n_{outputs}}}$

Source: Hands-On ML, Aurelien Geron

Summary: Weights initialization

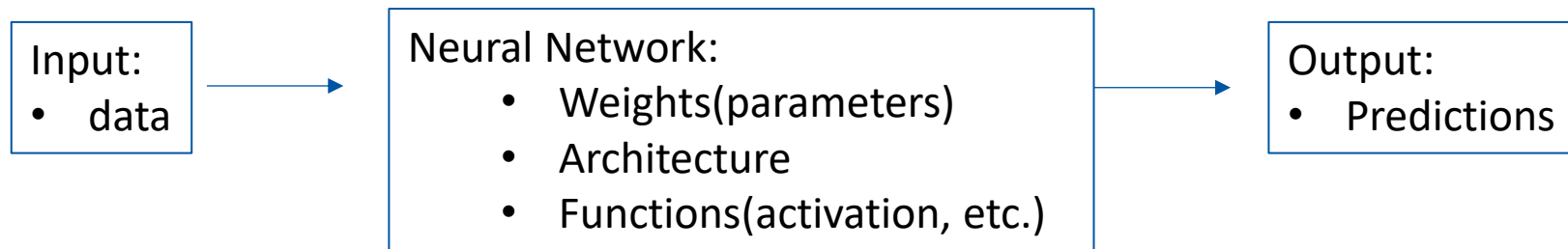
Activation function	Uniform distribution $[-r, r]$	Normal distribution
Logistic	$r = \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
Hyperbolic tangent	$r = \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
ReLU (and its variants)	$r = \sqrt{2} \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{2} \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$

Source: Hands-On ML, Aurelien Geron

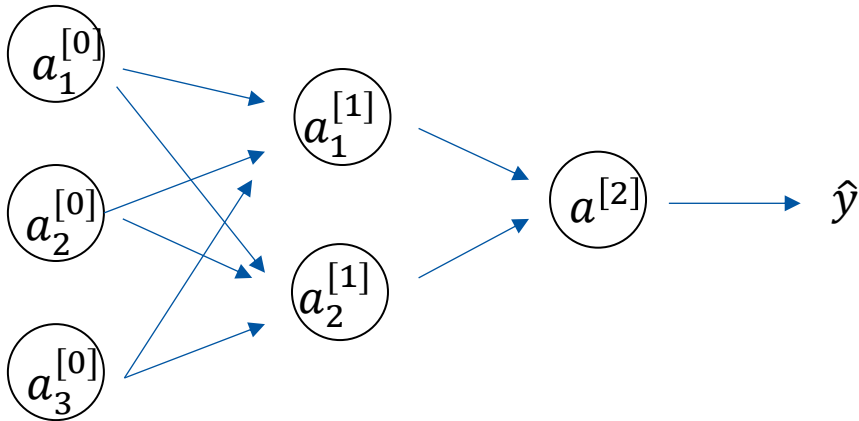
Forward propagation

- The inputs are fed into the neural network.
- This results in a forward cascade of computations across the layers, using the current set of weights.
- The final predicted output can be compared to the true output.

Forward propagation



Neural Network: 1 hidden layer



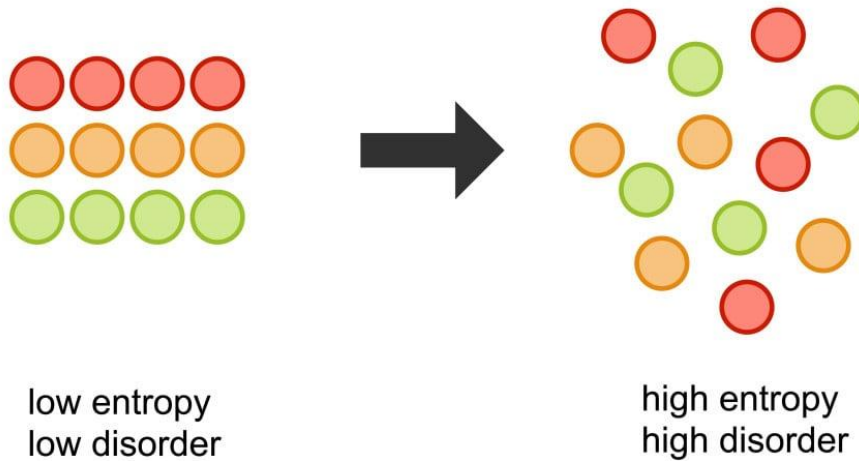
Input layer Hidden layer Output layer

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Entropy in thermodynamics

- Entropy is a concept of thermodynamics, introduced by Rudolf Clausius, a German physicist, in 1850.
- Entropy is a measurement of the **disorder or randomness of a system.**



Entropy in information

Claude Shannon introduced the concept of information entropy in his 1948 paper, “A Mathematical Theory of Communication”.

According to Shannon, **entropy** is the average number of bits required to represent or transmit an event drawn from the probability distribution for the random variable.

In other word, **the entropy** of a random variable is the average level of “information”.

In simple terms, entropy indicates the amount of uncertainty of an event.

Entropy in information

Information $h(x)$ can be calculated for an event x , given the probability of the event $P(x)$ as follows:

$$h(x) = -\log_2(P(x))$$

where $\log_2$ is the base-2 logarithm, meaning that the results are in bits.

For the entropy equation for the all events:

$$H(x) = -\sum_i P(x = i) \cdot \log_2(P(x = i))$$

Cross entropy

In the data science domain, the cross entropy between two discrete probability distributions is a metric that captures how similar the two distributions are.

Given the probabilities of the events from P and Q ,

Cross-entropy is following :

$$H(P, Q) = - \sum_x P(x) \cdot \log(Q(x))$$

where $P(x)$ is the probability of the event x in P , $Q(x)$ is the probability of event x in Q

$$H(y, \hat{y}) = - \sum_{j=0}^{N-1} y_j \log(\hat{y}_j)$$

Cross entropy in binary case

$$H(y, \hat{y}) = - \sum_{i \in \{0,1\}} y(i) \cdot \log(\hat{y}(i))$$

$$H(y, \hat{y}) = -[y_0 \cdot \log \hat{y}_0 + y_1 \cdot \log \hat{y}_1]$$

$$H(y, \hat{y}) = -[y \cdot \log \hat{y} + (1 - y) \cdot \log(1 - \hat{y})]$$

when $i = 1$; $y(i) = y, \hat{y}(i) = \hat{y}$

when $i = 0$; $y(i) = 1 - y, \hat{y}(i) = 1 - \hat{y}$

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

Cost function(binary cross entropy)

Cost function J

$$J = -\frac{1}{m} \sum_{i=1}^m [y^i \log(\hat{y}^i) + (1 - y^i) \log(1 - \hat{y}^i)]$$

$$\text{If } y = 1 \quad J = -\frac{1}{m} \sum_{i=1}^m [\log(\hat{y}^i)]$$

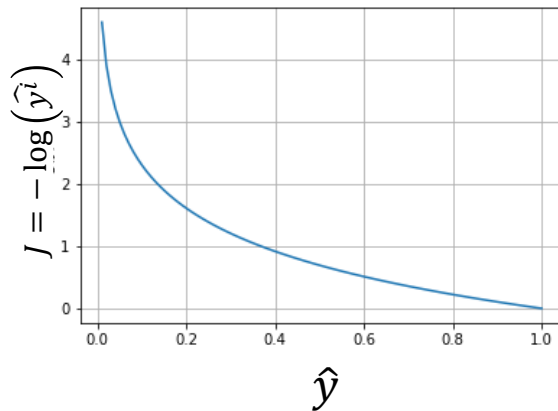
$$\text{If } y = 0 \quad J = -\frac{1}{m} \sum_{i=1}^m [\log(1 - \hat{y}^i)]$$

Cost function(binary cross entropy)

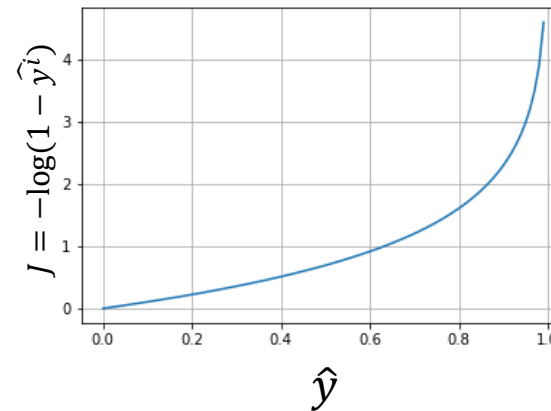
Given x , want $\hat{y}^i \approx y^i$

$$J = -[y^i \log(\hat{y}^i) + (1 - y^i) \log(1 - \hat{y}^i)]$$

If $y = 1$, $J = -\log(\hat{y}^i)$



If $y = 0$, $J = -\log(1 - \hat{y}^i)$



Accuracy in Classification

Accuracy

- *Sum(Prediction == Actual Y) / Total number of data*
- *mean(Prediction == Actual Y)*100*

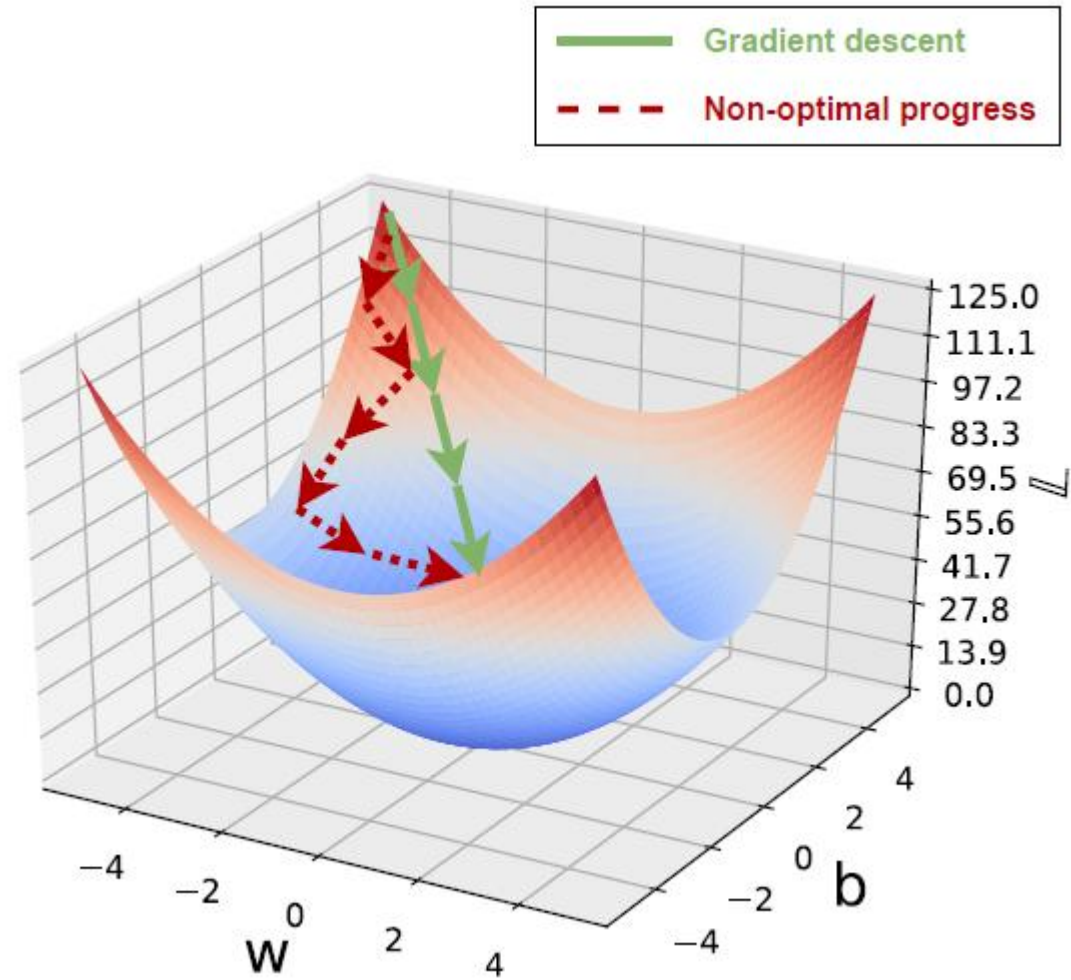
$$prediction = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

Gradient Descent

$$w = w - r \nabla_w L$$

$$\nabla_w L = \left[\frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k \right]$$

$$\partial w_{jk}^l := \partial w_{jk}^l - r \frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k$$



Source: Math and Architectures of Deep Learning, Krishnedu Chauhury

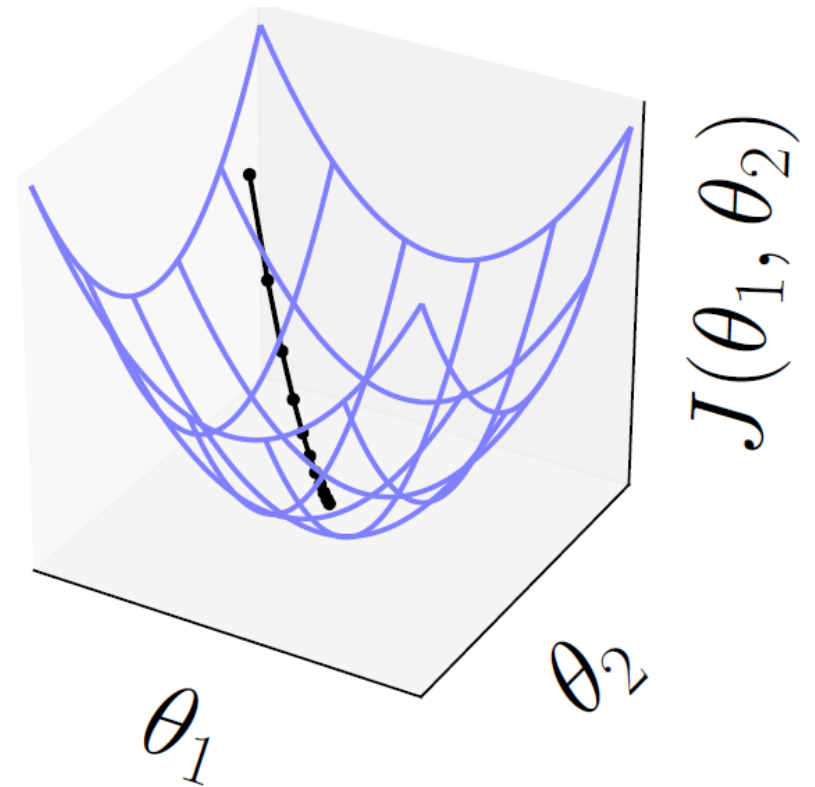
Gradient Descent

$$w = w - r \nabla_w L$$

$$\nabla_w L = \left[\frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k \right]$$

$$\partial w_{jk}^l := \partial w_{jk}^l - r \frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k$$

Source: Math and Architectures of Deep Learning, Krishnedu Chauhury



Source: "Adversarial Machine Learning," by Ian Goodfellow, ICLR Keynote, 2019

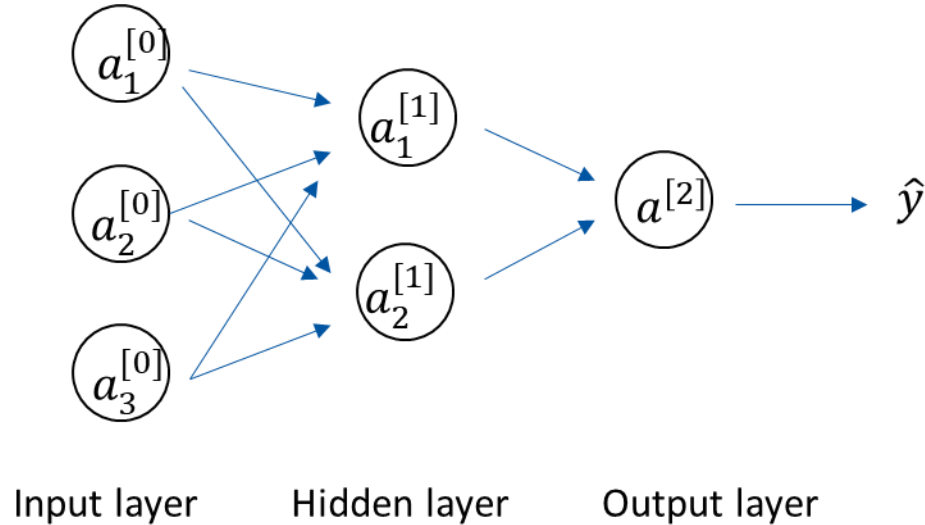
Gradient Descent

Find optimum W to minimize J

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

- W : weight or Model parameters
- J : cost function or loss function
- α : learning rate or step size
- $\frac{\partial J}{\partial W}$: gradient

Neural Network: 1 hidden layer

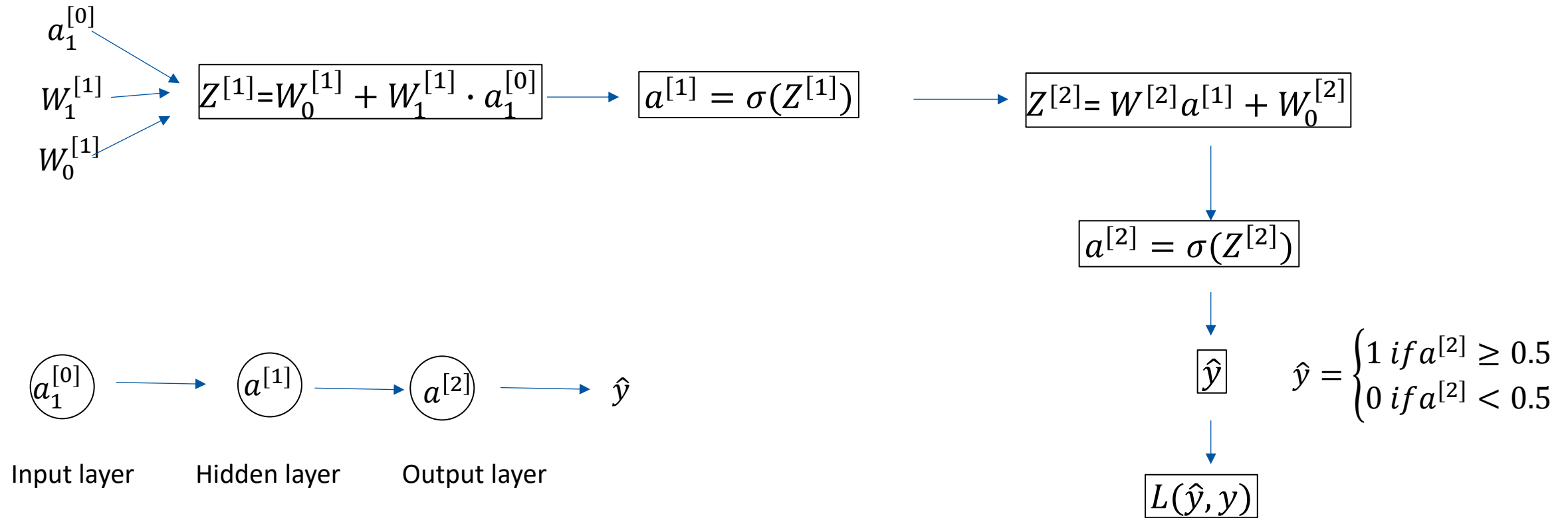


Parameters: $W^{[1]}, W^{[2]}$

Cost function: $J(W^{[1]}, W^{[2]})$

$$= \frac{1}{m} \sum_{i=1}^m L(\hat{y}, y)$$

Neural Network: 1 hidden layer



Forward propagation

$$Z^{[1]} = W^{[1]}a^{[0]} + W_0^{[1]}$$

$$a^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}a^{[1]} + W_0^{[2]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

